

Software Architecture and Design Specification

Project: Vehicle Parking System (VPS)
Version: 1.0
Authors: Team 10, Section C: Vivian Sobers E, D Harikrishnan, Dhruv Purushotham Raju, Gitanjali A
Date: 05-10-2026
Status: Submitted for Part-1 review
Course: Software Engineering (UE24CS341A)

Revision History

Version	Date	Author	Change Summary
1.0	05-10-2026	Team 10, Section C	First complete version: architecture, security architecture, sequence diagrams, API and data design

Approvals

Role	Name	Signature/Date
Integration and DevOps lead	Vivian Sobers E	
Database lead	D Harikrishnan	
Frontend lead	Dhruv Purushotham Raju	
Backend lead	Gitanjali A	
Course instructor		

1. Introduction

1.1 Purpose

This document describes the architecture and detailed design of the Vehicle Parking System (VPS), release 1.0. It explains how the system is divided into components, why that structure was chosen, how it meets the requirements in the SRS, and how the main flows, interfaces, errors and data are designed.

1.2 Scope

The scope is the whole system specified in the SRS: driver accounts and vehicles, slot search and reservation, slot allocation, gate check-in and check-out, billing, reports, the audit log and the occupancy forecast service. Deployment is covered at the level needed to explain security and availability; operating procedures are not.

1.3 Audience

The four developers of Team 10, the course instructor, testers writing integration and system tests, and anyone maintaining the system later.

1.4 Definitions

VPS: Vehicle Parking System. SRS: Software Requirements Specification. STP: Software Test Plan. RTM: requirement traceability matrix. DRF: Django REST Framework. HTMX: library for partial page updates. STRIDE: threat categories Spoofing, Tampering, Repudiation, Information disclosure, Denial of service and Elevation of privilege. Other terms follow Appendix A of the SRS.

2. Document Overview

2.1 How to use this document

Section 3 is the architecture: goals, the component diagram and its parts, the chosen pattern, technology, risks, traceability and security architecture. Section 4 is the design: sequence diagrams for the three main flows, the API, error handling and monitoring, user experience, the data model and open issues. Requirement IDs (FR, NFR, SEC, SAF) refer to the SRS.

2.2 Related Documents

- VPS Software Requirements Specification, version 1.0, 05-10-2026.
- VPS Software Test Plan, version 1.0, 05-10-2026.
- Requirement traceability matrix: Appendix C of the SRS.
- Diagram sources (PlantUML): docs/part-1/diagrams in <https://github.com/VivianSobers/Vehicle-Parking-System>

3. Architecture

3.1 Goals & Constraints

Goals, in priority order:

- Correct bookings and charges: no slot is ever held by two overlapping reservations (FR-RES-02), and every charge can be recomputed from stored data.
- Security: personal data and money-related records are protected, and every privileged action can be traced (SO-01 to SO-04).
- Fast gate operations: check-in and check-out within 2 seconds (NFR-PERF-02).
- A design small enough for four students to build, test and understand within one semester.

Constraints: Python for all server code; PostgreSQL 16; server-rendered pages with HTMX; GitHub, GitHub Actions and GitHub Projects for the process; no payment gateway; the forecast must never block booking (FR-FCST-03).

3.2 Stakeholders & Concerns

Stakeholder	Main concerns	Where addressed
Drivers	Finding a slot quickly; never losing a booking; fair charges; privacy	3.5, 3.9, 4.2 (Figure 2)
Gate Attendants	Speed at the barrier; working through outages	3.7, 4.2 (Figure 3), 4.5
Lot Administrators	Control of tariffs and slots; reports; accountability of staff	3.4, 3.9, 4.3
Development team	Clear module boundaries; one owner per area; testability	3.4, 3.5, 4.1
Course instructor	Traceability, security, use of SE practice	3.8, 3.9, SRS Appendix C

3.3 Component (UML) Diagram

Figure 1 shows the runtime components, the nodes they run on and their main dependencies. Domain modules call each other only through service functions; only the presentation layer handles HTTP.

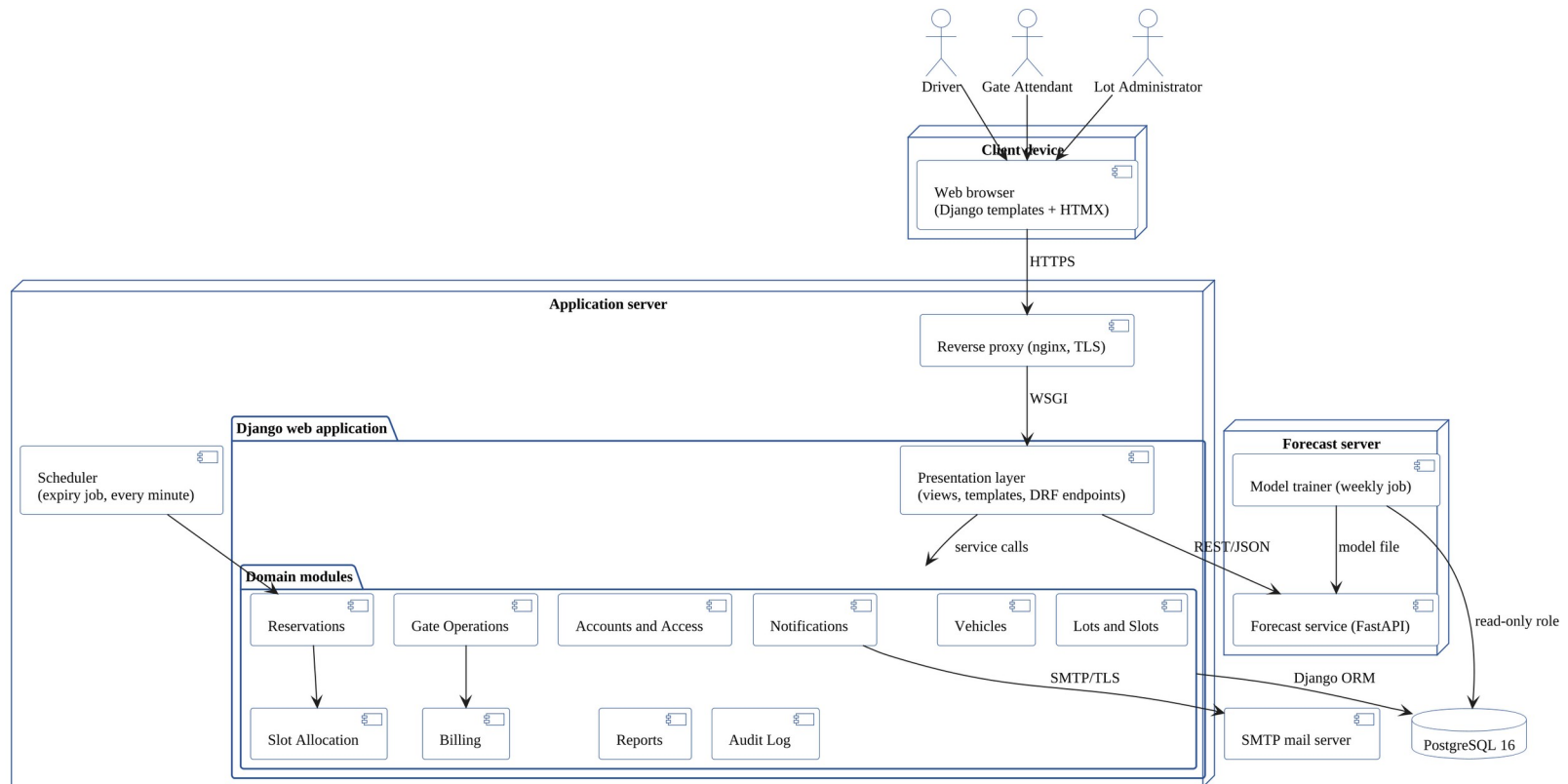


Figure 1: UML component diagram of VPS

3.4 Component Descriptions

Component	Responsibility	Owner
Presentation layer	Django views, templates and DRF endpoints; form validation; role checks on every request; HTMX partial updates.	Dhruv (templates), Gitanjali (views)
Accounts and Access	Registration, sign-in, logout, password reset, roles, staff accounts and the permission matrix.	Gitanjali
Vehicles	Vehicle registry and registration-number validation.	Gitanjali
Lots and Slots	Lots, slots, slot types, distance ranks and maintenance blocks; moving reservations off a blocked slot.	Hari
Reservations	Reservation state machine, booking rules, cancellation, expiry and history.	Gitanjali
Slot Allocation	Eligibility rules and the deterministic slot choice with retry on conflict.	Gitanjali, Hari
Gate Operations	Check-in, check-out, walk-in registration and manual time correction.	Gitanjali
Billing	Tariffs, charge computation, receipts, unpaid charges and payments.	Gitanjali
Reports	Occupancy and revenue reports, CSV export, audit log search.	Hari
Audit Log	Append-only record of privileged actions.	Hari
Notifications	Booking, cancellation, move and expiry e-mails sent asynchronously.	Vivian
Scheduler	Runs the expiry job every minute and the weekly model retraining.	Vivian
Forecast service	FastAPI service returning the predicted chance that a slot is free.	Vivian
Model trainer	Weekly job that builds the forecast model from completed reservations through a read-only database role.	Vivian
PostgreSQL database	System of record; enforces the no-double-booking exclusion constraint.	Hari

Each domain module is a Django app with a `services.py` holding its business rules, so the rules can be unit-tested without HTTP (NFR-TST-01). Modules depend on each other only in the directions shown in Figure 1; for example Reservations calls Slot Allocation, never the other way round.

3.5 Chosen Architecture Pattern and Rationale

Pattern: a layered modular monolith with one separate service. The Django application is one deployable unit with three layers: presentation (views, templates, DRF endpoints),

domain services (one module per business area) and data access (Django ORM on PostgreSQL). The forecast runs as a separate FastAPI service.

Reasons:

- One database transaction can cover a whole booking or check-out, which keeps the double-booking and billing rules simple to get right. Splitting these into microservices would need distributed transactions for no benefit at this scale.
- The double-booking guarantee is placed in the database as an exclusion constraint, so it holds even under concurrent requests and even if application code has a bug.
- The forecast service is separate because it has a different workload (model loading, numerical libraries) and a different failure mode: if it fails, booking must continue (FR-FCST-03).
- Module boundaries follow team ownership, so four people can work in parallel with few merge conflicts.

Alternative	Why it was not chosen
Microservices for every module	Distributed transactions for booking and billing; too much operational work for a four-person team.
Single-page app (React) plus REST API	A second language and build pipeline; the UI does not need rich client state.
Locking in application code only (no exclusion constraint)	Easy to get wrong under concurrency; a missed code path could double-book.

3.6 Technology Stack & Data Stores

Layer	Technology
Client	Django templates, HTMX 2.x, plain CSS; no client-side framework
Web application	Python 3.12, Django 5.x, Django REST Framework 3.15, Gunicorn
Forecast	FastAPI, scikit-learn, pandas; model stored as a versioned file on the forecast server
Data store	PostgreSQL 16 with btree_gist; one database, separate read-only role for the model trainer
Scheduling	cron on the application server running Django management commands (expiry every minute, retraining weekly)
Edge	nginx reverse proxy terminating TLS
E-mail	SMTP with STARTTLS, sent from a background worker so requests do not wait for the mail server
Process and CI	GitHub, GitHub Projects, GitHub Actions (ruff, pytest with coverage, secret scanning)

Planned repository layout:

```
vps/          Django project
accounts/  vehicles/  lots/  reservations/  allocation/
```

gate/	billing/	reports/	audit/	notifications/
templates/		shared	page	templates
forecast/app/		FastAPI	service	
forecast/train/		weekly	training	job
tests/		pytest	suites	
deploy/nginx/		proxy	configuration	
docs/		deliverables	and	diagrams

3.7 Risks & Mitigations

Risk	Mitigation
Two drivers book the last slot at the same moment.	PostgreSQL exclusion constraint plus retry on the next candidate (FR-ALLOC-03); tested with 50 parallel requests (TC-07).
An overstaying vehicle still occupies the slot of the next booking.	Check-in reallocates to another free slot (FR-GATE-07); overstay is billed at 1.5 times the rate to discourage it.
The application or network is down while vehicles queue.	Paper fallback and later time correction with audit (FR-GATE-06, SAF-03).
The expiry job stops running, so slots stay held.	Job writes a heartbeat; an alert fires if it is more than 5 minutes old (Section 4.4).
No historical data for the forecast.	Baseline from simulated arrivals, clearly labelled "Estimate" (FR-FCST-02).
Forecast service slow or down.	2-second timeout and fallback message; search never depends on it (FR-FCST-03).
Team members unfamiliar with Django.	Module templates and pair review on the first pull request in each module.

3.8 Traceability to Requirements

Table 1 maps each SRS feature to the components that implement it. The full per-requirement matrix, including test cases, is Appendix C of the SRS.

Table 1: Features to components

SRS feature	Requirements	Components
5.1 User Accounts and Access Control	FR-AUTH-01 to FR-AUTH-06	Accounts and Access, Presentation layer
5.2 Vehicle Registry	FR-VEH-01 to FR-VEH-04	Vehicles
5.3 Lot and Slot Management	FR-LOT-01 to FR-LOT-04	Lots and Slots, Notifications
5.4 Slot Availability Search	FR-SRCH-01 to FR-SRCH-03	Slot Allocation, Presentation layer
5.5 Reservation Management	FR-RES-01 to FR-RES-07	Reservations, Slot Allocation, Scheduler, Notifications, PostgreSQL database
5.6 Slot Allocation	FR-ALLOC-01 to FR-ALLOC-03	Slot Allocation, PostgreSQL database

SRS feature	Requirements	Components
5.7 Gate Check-in and Check-out	FR-GATE-01 to FR-GATE-07	Gate Operations, Billing, Audit Log
5.8 Pricing and Billing	FR-BILL-01 to FR-BILL-05	Billing
5.9 Reports and Audit	FR-RPT-01 to FR-RPT-04	Reports, Audit Log
5.10 Occupancy Forecast	FR-FCST-01 to FR-FCST-03	Forecast service, Model trainer, Scheduler

Table 2: Non-functional requirements to mechanisms

Non-functional requirement	Architectural mechanism
FR-RES-02, NFR-REL-01 (no double booking)	Exclusion constraint on (slot, period) for Reserved and Active rows; retry in Slot Allocation
NFR-PERF-01 (search under 1 s)	GiST index on reservation period; one query per search; HTMX partial responses
NFR-PERF-02 (gate under 2 s)	Single transaction per gate action; e-mail sent asynchronously
FR-FCST-03 (forecast fallback)	Separate service, 2-second client timeout, graceful message
SEC-03, SEC-07 (access control and audit)	Permission check in the presentation layer; Audit Log module with insert-only table
NFR-MNT-01, NFR-MNT-02 (maintainability)	services.py per module; CI gates on coverage, ruff and review

3.9 Security Architecture

Trust boundaries. (1) Internet to nginx: only HTTPS is accepted. (2) nginx to the Django application: private network. (3) Application to PostgreSQL and to the forecast service: private network, separate credentials. (4) Model trainer to PostgreSQL: read-only database role, so a compromised trainer cannot change bookings or charges.

Access control. Every view declares the roles allowed to use it; a shared decorator rejects other roles with HTTP 403 and logs the attempt (SEC-03). The role matrix is SRS Appendix B.3. Object-level checks let a Driver see and cancel only their own reservations.

Table 3: STRIDE threat model

STRIDE threat	Example in VPS	Mitigation	Requirement
Spoofing	Guessing a Driver's password	PBKDF2 hashes, lockout after 5 failures, rate limit per IP	SEC-01, SEC-02
Tampering	Changing another driver's booking or a tariff through crafted requests	Server-side role and ownership checks, CSRF tokens, ORM queries only	SEC-03, SEC-05, SEC-06
Repudiation	Attendant denies waiving a charge or editing a time	Insert-only audit log with actor, old and new values	SEC-07
Information	Reading mobile numbers	TLS, Secure and HttpOnly cookies,	SEC-04, SEC-08,

STRIDE threat	Example in VPS	Mitigation	Requirement
disclosure	or session cookies	masking for attendants, no secrets in logs	SEC-09
Denial of service	Flooding sign-in or search	nginx and application rate limits; lockout scoped to one account	SEC-02
Elevation of privilege	Driver calling an administrator endpoint	Role decorator on every view; staff accounts created only by administrators	SEC-03, FR-AUTH-03

Secrets and supply chain. Secrets come from environment variables (SEC-10). CI runs a secret scanner and pip-audit on every pull request, and dependencies are pinned in requirements files.

4. Design

4.1 Design Overview

Each request follows the same path: the view authenticates the user, checks the role, validates input with a Django form or DRF serializer, calls one service function and renders the result. Service functions own the business rules and the transaction boundary; they take plain values and return model objects or raise a domain error (Section 4.4). Time is passed in explicitly so that rules such as expiry and the check-in window can be tested with a fixed clock.

Design decisions:

- Allocation is a pure function over a candidate query; the database constraint is the final guard against races.
- Charges are separate rows (Parking, Overstay, LateCancel, NoShow), so a receipt is the sum of its rows and any line can be audited.
- Reservations store a time range (`tstzrange`) instead of two columns, which lets PostgreSQL index and constrain overlaps directly.

4.2 UML Sequence Diagrams

Three flows are shown: making a reservation (Figure 2), checking a vehicle out (Figure 3) and signing in (Figure 4).

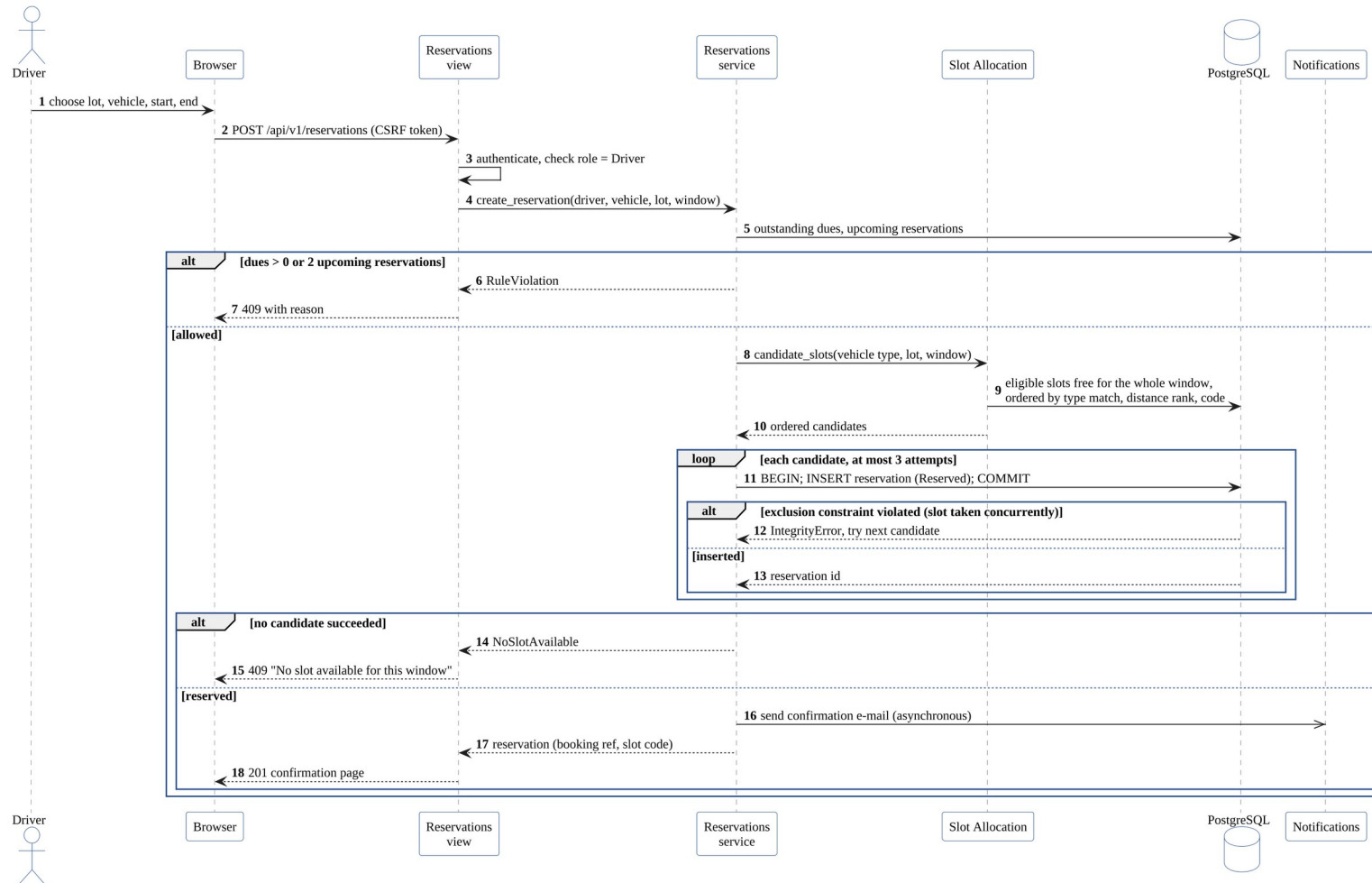


Figure 2: Sequence diagram: reserve a slot, including the retry when a concurrent booking takes the chosen slot

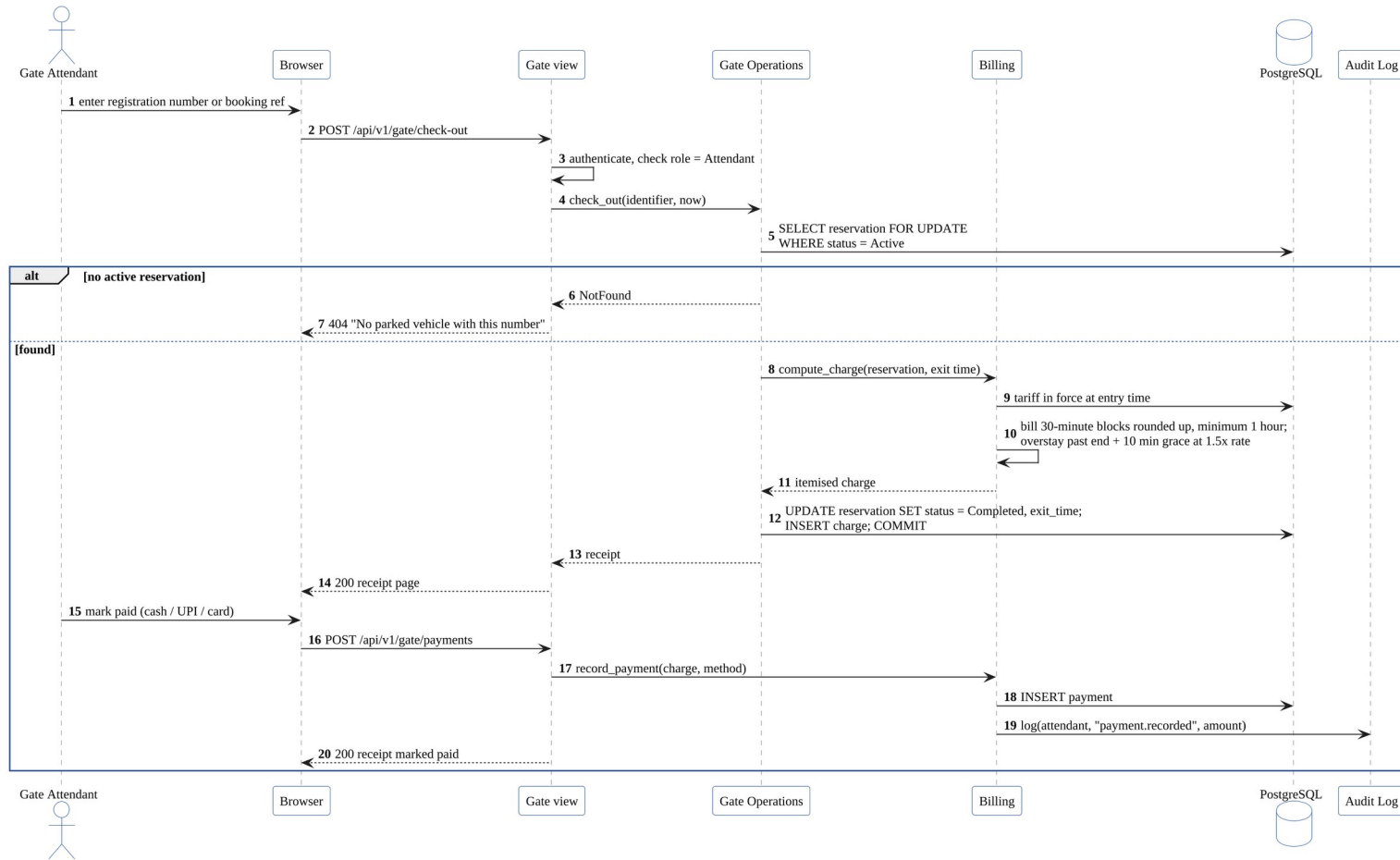


Figure 3: Sequence diagram: check out a vehicle, compute the charge and record payment

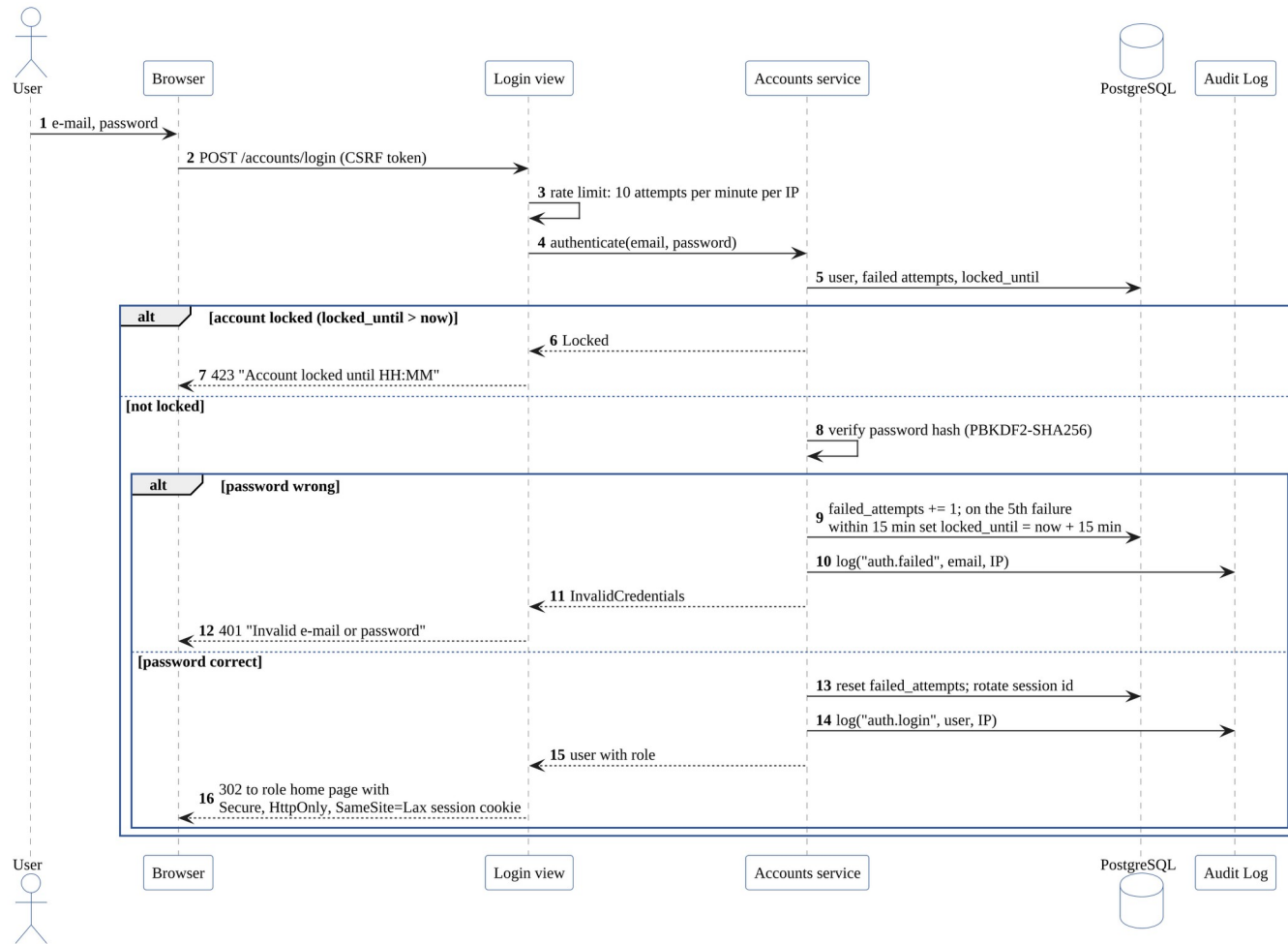


Figure 4: Sequence diagram: sign in with lockout and audit

4.3 API Design

JSON endpoints are served by DRF under `/api/v1/` and used by the HTMX pages and by tests. All require a signed-in session and a CSRF token for state changes. Errors use the format in Section 4.4.

Reservations component (role: Driver)

Endpoint	Method	Request	Response	Errors
<code>/api/v1/availability</code>	GET	<code>lot_id, vehicle_id, start, end</code>	<code>{eligible_free: int, by_type: {...}}</code>	400 invalid window (rule named)
<code>/api/v1/reservations</code>	POST	<code>{lot_id, vehicle_id, start, end}</code>	201 <code>{booking_ref, slot_code, start, end, state}</code>	409 no slot, limit reached or unpaid charge; 400 validation
<code>/api/v1/reservations</code>	GET	page (20 per page)	<code>{results: [...], next, previous}</code>	-
<code>/api/v1/reservations/{ref}/cancel</code>	POST	-	200 <code>{state: Cancelled, charge: amount or null}</code>	404 not yours; 409 already started

Gate component (role: Gate Attendant)

Endpoint	Method	Request	Response	Errors
<code>/api/v1/gate/check-in</code>	POST	<code>{identifier}</code>	200 <code>{booking_ref, slot_code, entry_time}</code>	404 not found; 409 too early, with earliest time
<code>/api/v1/gate/walk-in</code>	POST	<code>{registration_no, vehicle_type}</code>	201 <code>{booking_ref, slot_code}</code>	409 no slot until closing; 400 invalid number
<code>/api/v1/gate/check-out</code>	POST	<code>{identifier}</code>	200 <code>{receipt: {lines, total, ...}}</code>	404 no active stay
<code>/api/v1/gate/payments</code>	POST	<code>{charge_id, method}</code>	201 <code>{payment_id, paid_at}</code>	409 already paid; 400 bad method

Forecast service (internal, called by the web application)

Endpoint	Method	Request	Response	Errors
<code>/v1/forecast</code>	GET	<code>lot_id, vehicle_type, arrival (ISO 8601)</code>	<code>{probability: 0..1, band: Low Medium High, estimate: bool, model_version}</code>	422 invalid input; client treats timeout above 2 s as unavailable
<code>/v1/health</code>	GET	-	<code>{status: ok, model_version, trained_at}</code>	503 model not loaded

Example request and response for creating a reservation:

```
POST /api/v1/reservations
{"lot_id": 1, "vehicle_id": 7,
```

```
"start": "2026-10-07T09:30:00+05:30", "end": "2026-10-07T11:30:00+05:30"}
```

201 Created

```
{"booking_ref": "VPS-261007-0042", "slot_code": "B-14",  
  "start": "2026-10-07T09:30:00+05:30", "end": "2026-10-07T11:30:00+05:30",  
  "state": "Reserved"}
```

4.4 Error Handling, Logging & Monitoring

Domain errors. Services raise typed errors (ValidationFailed, RuleViolation, NoSlotAvailable, NotFound, PermissionDenied, Locked). One exception handler maps them to HTTP status codes and a standard body; HTML pages show the same message next to the relevant field or in a banner.

```
{"error": {"code": "NO_SLOT_AVAILABLE",  
          "message": "No slot available for this window",  
          "details": {"lot_id": 1}}}
```

Error	HTTP status	Shown to user
ValidationFailed	400	Field message, for example "Search window must be at least 30 minutes"
Not signed in / PermissionDenied	401 / 403	"Please sign in" / "You do not have access to this page"
NotFound	404	"No parked vehicle with this number"
RuleViolation, NoSlotAvailable	409	The rule that blocked the action
Locked	423	"Account locked until HH:MM"
Unexpected error	500	"Something went wrong. Reference: <request id>"; details only in logs

Logging. Structured JSON logs with timestamp, level, request id, user id and role, path and duration. Passwords, tokens and full mobile numbers are never logged (SEC-09). Logs are kept for 30 days; the audit log is separate and kept for 1 year (SEC-07).

Monitoring. /healthz on the web application checks the database; /v1/health on the forecast service. Metrics and alerts: 95th percentile response time per endpoint against NFR-PERF targets; rate of NoSlotAvailable and IntegrityError retries; expiry job heartbeat (alert if older than 5 minutes); failed e-mail count; uptime between 06:00 and 23:00 for NFR-AVL-01.

4.5 UX Design

- Booking takes at most 4 screens: search, choose window, confirm, confirmation (NFR-USE-01).
- The gate console is one screen with a single identifier field, large buttons for Check in, Check out and Walk-in, and keyboard-only operation.
- Slot types use both colour and a text label, so colour is never the only signal; contrast meets WCAG 2.1 AA (NFR-USE-02).
- The forecast panel shows a percentage and a Low, Medium or High band, with "Estimate" when the baseline model is used.

- Layouts are mobile first, from 360 pixels wide, because most Drivers book from phones.

4.6 Data Design

Figure 5 shows the main entities. Money is stored as `numeric(8,2)` in rupees; times as `timestampz` in UTC.

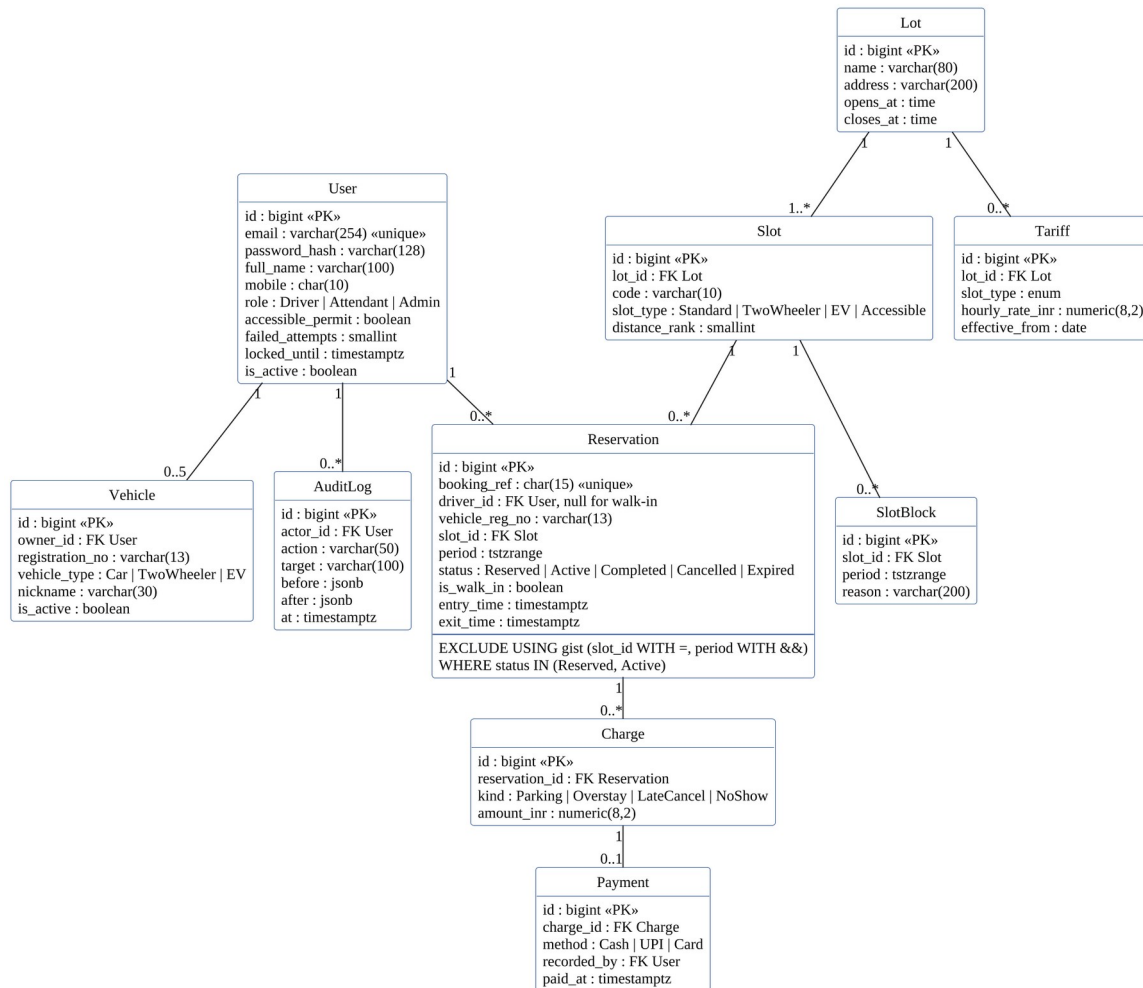


Figure 5: Data model (UML class diagram)

The rule that no slot is held twice is enforced by this constraint (Hari owns the migration):

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE reservation ADD CONSTRAINT no_overlapping_holds
    EXCLUDE USING gist (slot_id WITH =, period WITH &&)
    WHERE (status IN ('Reserved', 'Active'));
```

Other constraints and indexes: unique (lot_id, code) on slot; unique booking_ref; partial unique index on vehicle registration_no where is_active; check constraints for positive distance_rank and amounts; GiST index on slot_block.period. The audit_log table grants INSERT and SELECT only to the application role (SEC-07).

4.7 Open Issues & Next Steps

- Decide whether e-mail goes through a task queue (Celery) or a simple database outbox polled by cron; the outbox is the current plan.
- Choose the hosting for the demo deployment and confirm TLS certificates.
- Define the simulated-arrival generator used for the forecast baseline and test data.
- Future work outside this release: online payment, number-plate recognition at the gate, SMS notifications.

5. Appendices

5.1 Glossary

See Section 1.4 and SRS Appendix A.

5.2 References

- ISO/IEC/IEEE 42010:2022, Systems and software engineering: Architecture description.
- OWASP Application Security Verification Standard 4.0.3.
- Microsoft, The STRIDE Threat Model.
- PostgreSQL 16 documentation, Exclusion Constraints and the btree_gist extension.
- Django 5 and Django REST Framework documentation.

5.3 Tools

PlantUML for all diagrams (sources in docs/part-1/diagrams), GitHub Projects for planning, GitHub Actions for CI.